

Ödev 13: Bir Kural Motoru

Birleştirilmiş predicate'lerden uygunluk kuralları kurun, dört metot referansı biçimini de kullanın, sonra bilerek yakalama kuralına çarpın.

Veri

```
record Applicant(String name, int age, int creditScore, boolean employed) { }

List.of(
    new Applicant("ada", 36, 780, true),
    new Applicant("grace", 45, 610, true),
    new Applicant("alan", 17, 800, false),
    new Applicant("edsger", 52, 550, true),
    new Applicant("barbara", 29, 720, false));
```

Birinci Bölüm: Birleştirilmiş Predicate'ler

`RulesEngine.java` dosyasını klasik biçimde oluşturun.

Ayrı ayrı adlandırılmış üç predicate yazın:

Ad	Kural
<code>isAdult</code>	yaş 18 ve üzeri
<code>hasGoodCredit</code>	kredi puanı 700 ve üzeri
<code>isEmployed</code>	employed true

Sonra bunları **birleştirerek**, yeni bir lambda yazmadan iki bileşik kural kurun:

- `eligible` — yetişkin, iyi kredi ve çalışıyor
- `needsReview` — yetişkin, ama kredi 700 altında

Her başvuran için üç sonucu ve ayrıca reşit olup olmadığını gösteren bir satır yazdırın; sonuncusu için `negate()` kullanın.

Üç adlandırılmış predicate'in neden tek bir büyük lambda'yı yendiğini bir yorumda yazın. Cevabınız tekrar kullanımdan bahsetmeli: iki bileşik kuraldan biri diğeriyle bir bileşen paylaşıyor.

İkinci Bölüm: Dört Metot Referansı Biçimi

Her biçimi en az bir kez kullanın, her birinde hangisi olduğunu isimlendiren bir yorumla:

1. Bir **statik** metot referansı

2. Bir **kurucu** referansı
3. Bir **bağlı örnek** referansı
4. Bir **bağısız örnek** referansı

Sonra Zor Kısım Açıklayın

Üçüncü ve dördüncü biçimlerin ikisi de `Something::method` olarak yazılır. Kendi kelimelerinizle şunları açıklayan bir yorum yazın:

- Her durumda `::` işaretinin solunda ne durduğu ve bunun neden ayırt edici işaret olduğu
- Hangisinin alıcısını bir kez seçtiği, hangisinin her çağrıda seçtiği
- Argüman sayısı açısından gözlemlenebilir sonuç.** Tek argümanlı bir metoda bağlı referans, kaç parametrelili bir fonksiyonel arayüz ister? Aynı metoda bağısız referans için ne dersiniz? İkisini de gösterin.

Son olarak `andThen` ile bir `Applicant` nesnesini A, B veya C kredi bandına çeviren bir boru hattı kurun ve herkes için yazdırın.

Üçüncü Bölüm: Yakalama Kuralı

Önce **derlenmeyen** sürümünü yazın: bir `forEach` içinde artırılan yerel bir `int approved = 0;`. Hatayı doğrulayın, sonra kuralın neden var olduğunun açıklamasıyla birlikte bir yoruma koyun.

Sonra aynı sayımı çalışan **üç** biçimde yazın:

1. `int[] box = {0}` numarası
2. Sıradan bir `for` döngüsü
3. `filter` ve `count` kullanan bir stream

Soru

Düz `int` derlenmezken dizi numarasının neden derlendiğini ve yine de neden kötü bir cevap olduğunu açıklayan bir yorum yazın. Açıklamanız **değişken** ile **gösterdiği nesne** arasında ayırım yapmalı, çünkü kuralın tamamı o ayırım.

Bir lambda içinde bir `List` yakalayıp değiştirerek bunun yasal olduğunu gösterip bitirin, sonra yine de `.toList()` kullanmanın neden daha iyi olduğunu söyleyin.

Kabul Kriterleri

- ☐ `java RulesEngine.java` ile çalışıyor
- ☐ Ayrı ayrı adlandırılmış üç predicate var
- ☐ `eligible` ve `needsReview` yeniden yazılarak değil birleştirilerek kuruldu

- ☐ `negate()` kullanılıyor
- ☐ Bir yorum birleştirmeyi, paylaşılan bileşenden bahsederek gerekçelendiriyor
- ☐ Dört metod referansı biçimi de görünüyor, her biri etiketli
- ☐ Bağlı ile bağısız açıklaması argüman sayısı sonucunu kapsıyor ve her iki imzayı gösteriyor
- ☐ Bir `andThen` boru hattı kredi bantları üretiyor
- ☐ Derlenmeyen yakalama denemesi gerçek hatasıyla alıntılanmış
- ☐ Sayım üç çalışan biçimde üretiliyor, hepsi aynı sonucu veriyor
- ☐ Açıklama değişkeni nesneden ayırıyor

İleri Seviye

Bir `Predicate<Applicant>` fabrikası yazın: minimum bir puan alan ve ona karşı test eden bir predicate döndüren bir metod.

Sonra o metodun neyi yakaladığını, yakalanan parametrenin etkin final olup olmadığını, ve bu modülün yığın çerçeveleri hakkında söyledikleri göz önüne alındığında bir metod parametresinin neden yakalanmasına izin verildiğini bir yorumda yazın.

İpucu, `compose` tip denetiminden geçmiyorsa

`a.andThen(b)` önce `a` çalıştırır. `a.compose(b)` önce `b` çalıştırır. `a` bir `Applicant` alıp `b` bir `int` alırken yalnızca bir sıralama tutar. Derleyici reddediyorsa ters yazmışsınız, ve o ret size iyilik yapıyor.